



‘Machine Learning’

Pablo Blas Hernández

El objetivo de esta práctica es probar y validar diferentes algoritmos de clasificación binaria para ver cuál funciona mejor a la hora de predecir qué clientes se van a dar de baja. Utilizando un conjunto de datos que integra 20 variables independientes (V1 a V20), las cuales capturan desde dimensiones geográficas y antigüedad de la cuenta hasta métricas granulares de consumo y comportamiento en el servicio de atención al cliente, se busca predecir con precisión la variable dependiente Y (riesgo de abandono).

1. Análisis y depuración de datos

1.1. Carga y exploración inicial

El primer paso es cargar el dataset y examinar su estructura, dimensiones y tipología de variables.

```
1 df = pd.read_csv('BBDD_ML_TAREA.csv')
2 print(f"Dimensiones: {df.shape}")
3 print(f"\nTipos de datos:")
4 print(df.dtypes)

Dimensiones: (9200, 21)

Tipos de datos:
V1      int64
V2      int64
V3     float64
V4      int64
V5      int64
V6      int64
V7      int64
V8     float64
V9      int64
V10     float64
V11     float64
V12     int64
V13     float64
V14     float64
V15     float64
V16     float64
V17     float64
V18     int64
V19     float64
V20     int64
Y       int64
dtype: object
```

El dataset contiene 9.200 filas y 21 columnas. Al mirar los tipos de datos, llama la atención que las variables V3 y V15 aparecen como float cuando deberían ser enteras (V3 es código de área y V15 es número de llamadas). En pandas, cuando una columna entera tiene valores nulos se promueve automáticamente a float64, así que esto está diciendo donde hay *missing values* antes incluso de buscarlos.

Se utiliza V4 (teléfono) como clave para identificar y eliminar duplicados reales (mismo cliente registrado varias veces), diferenciándolos de clientes distintos con patrones de consumo idénticos. Una vez depurado el dataset, se descarta V4 para evitar ruido, ya que, al ser un identificador único, carece de poder predictivo.

1.2. Análisis de duplicados

```
1 # 1) ¿Hay números de teléfono que aparecen más de una vez?
2 tel_counts = df['V4'].value_counts()
3 tel_rep = tel_counts[tel_counts > 1]
4 print(f"Teléfonos únicos: {df['V4'].nunique()}")
5 print(f"Teléfonos repetidos (aparecen >1 vez): {len(tel_rep)}")
6 print(f"Filas afectadas por repeticiones: {tel_rep.sum()}")
7
8 # 2) ¿Esas repeticiones son filas idénticas o clientes que cambiaron?
9 # Agrupamos por V4 y miramos si todas las filas del mismo teléfono son iguales
10 duplicados_reales = 0
11 duplicados_distintos = 0
12
13 for tel, grupo in df[df['V4'].isin(tel_rep.index)].groupby('V4'):
14     if grupo.drop(columns='V4').drop_duplicates().shape[0] == 1:
15         duplicados_reales += len(grupo) - 1
16     else:
17         duplicados_distintos += len(grupo)
18
19 print(f"\nFilas duplicadas reales (mismo tel, mismo patrón): {duplicados_reales}")
20 print(f"Filas con mismo tel pero distinto patrón: {duplicados_distintos}")
21
22 # 3) Verificación cruzada con duplicated() genérico
23 n_dup_generico = df.duplicated().sum()
24 print(f"\nFilas duplicadas (comprobación genérica todas las columnas): {n_dup_generico}")
25
26 # 4) Distribución de Y antes y después
27 print(f"\n--- Distribución de Y CON duplicados ---")
28 print(df['Y'].value_counts())
29
30 df_sin_dup = df.drop_duplicates()
31 print(f"\n--- Distribución de Y SIN duplicados ---")
32 print(df_sin_dup['Y'].value_counts())
33 print(f"\nProporción clase 1 sin duplicados: {df_sin_dup['Y'].mean()*100:.1f}%")
```

Teléfonos únicos: 3536
Teléfonos repetidos (aparecen >1 vez): 1921
Filas afectadas por repeticiones: 7585

Filas duplicadas reales (mismo tel, mismo patrón): 5652
Filas con mismo tel pero distinto patrón: 14

Filas duplicadas (comprobación genérica todas las columnas): 5662

--- Distribución de Y CON duplicados ---

Y	count
0	4600
1	4600

Name: count, dtype: int64

--- Distribución de Y SIN duplicados ---

Y	count
0	2832
1	706

Name: count, dtype: int64

Proporción clase 1 sin duplicados: 20.0%

El dataset contiene 5.662 filas duplicadas. La decisión es eliminar los duplicados, y posteriormente, proceder a eliminar la variable V4 tras haber completado el filtrado de duplicados, evitando así la introducción de ruido en el modelo.

```
1 # Eliminamos duplicados y trabajamos con los datos reales
2 df = df.drop_duplicates().reset_index(drop=True)
3 print(f"Dimensiones tras eliminar duplicados: {df.shape}")
4 print(f"Distribución de Y: {df['Y'].value_counts().to_dict()}")
```

Dimensiones tras eliminar duplicados: (3538, 21)
Distribución de Y: {0: 2832, 1: 706}

```
# Eliminamos V4: identificador sin valor predictivo
df.drop(columns=['V4'], inplace=True)
```

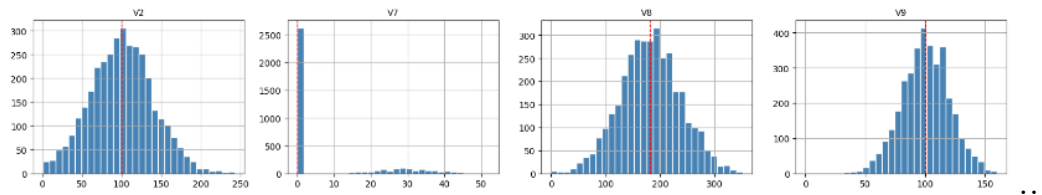
Tras eliminarlas, la distribución de Y pasa a 80/20 (2.832 no-abandono vs. 706 abandono). Esto indica que se aplicó un *oversampling* (duplicación de filas, sobre todo de la clase minoritaria) para balancear las clases. Un desbalanceado 80/20 que hay que tenerlo en cuenta a la hora de modelar, porque un clasificador que prediga siempre "no abandona" sin mirar ninguna variable ya acertaría en los 2.832 de la clase mayoritaria, lo que da un *accuracy* de $2.832 / 3.538 = 0.80$, es decir un 80%. Ese 80% sería completamente inútil porque el modelo no habría aprendido nada, simplemente repetiría la clase más frecuente y no detectaría ni un solo cliente que se va a ir. Es algo que tendré en cuenta más adelante a la hora de elegir la métrica de evaluación y de configurar los modelos.

1.3. Identificación y reclasificación de variables

Aunque todas las columnas llegan en formato numérico, hay variables que no tienen sentido aritmético. V1 (estado/región) y V3 (código de área) son etiquetas geográficas sin orden: decir que el área 510 es mayor que el área 408 no tiene ningún sentido, y si el modelo las trata como numéricas aprenderá relaciones falsas. V5 y V6 son indicadores binarios (plan internacional y buzón de voz). El resto (V2, V7-V20) sí son conteos y medidas continuas con sentido aritmético. Se reclasifican V1, V3, V5 y V6 como categóricas. V1 presenta 51 categorías únicas, V3 tiene 3 valores y V5/V6 son binarias.

1.4. Análisis descriptivo

Se han generado histogramas, boxplots y diagramas de barras de todas las variables. Las variables de minutos (V8, V11, V14, V17) y costes (V10, V13, V16, V19) presentan distribuciones simétricas. V7 tiene un pico marcado en cero (la mayoría no usa buzón de voz). V20 está muy concentrada en valores bajos (mediana=1, máximo=9) con una cola derecha pronunciada. V2 (antigüedad) se distribuye de forma bastante uniforme entre 1 y 243. V1 tiene 51 categorías con frecuencias desiguales y V3 solo 3 valores.



1.5. Análisis de valores perdidos

```
1 missing = df.isnull().sum()
2 missing = missing[missing > 0]
3 missing_pct = (missing / len(df)) * 100
4
5 print("--- Valores nulos ---")
6 for col in missing.index:
7     print(f" {col}: {missing[col]} ({missing_pct[col]:.2f}%)")
8 print(f"\nTodas las variables con nulos están por debajo del 2%.")
9 print("Se imputan con mediana (numéricas) y moda (categóricas).")

--- Valores nulos ---
V3: 35 (0.99%)
V10: 13 (0.37%)
V14: 26 (0.73%)
V15: 28 (0.79%)
V19: 16 (0.45%)

Todas las variables con nulos están por debajo del 2%.
Se imputan con mediana (numéricas) y moda (categóricas).
```

Tras eliminar duplicados, encontramos nulos en 5 variables, todos por debajo del 2%. Las SVM no gestionan datos incompletos. Al ser tan pocos, la imputación es más adecuada que eliminar filas (que reduciría el tamaño muestral y podría introducir sesgo). Para las numéricas uso la mediana (robusta frente a *outliers*) y para las categóricas la moda (la media de códigos de área no tendría sentido).

1.6. Análisis de valores atípicos

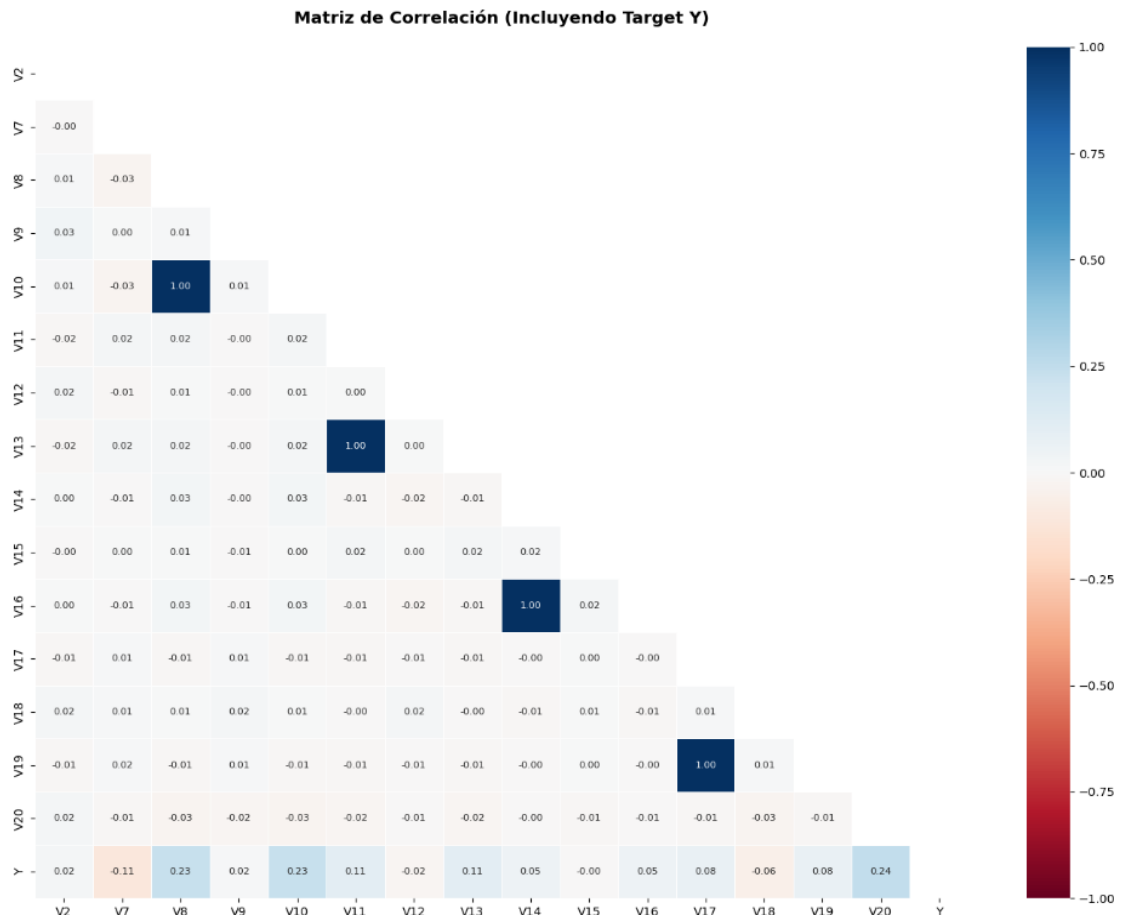
```
1 # Detección cuantitativa mediante IQR x 1.5
2 print("Detección estadística (IQR x 1.5):")
3 for col in numerical_cols:
4     Q1, Q3 = df[col].quantile(0.25), df[col].quantile(0.75)
5     IQR = Q3 - Q1
6     n_out = ((df[col] < Q1 - 1.5*IQR) | (df[col] > Q3 + 1.5*IQR)).sum()
7     if n_out > 0:
8         print(f" {col}: {n_out} outliers ({n_out/len(df)*100:.2f}%)")
```

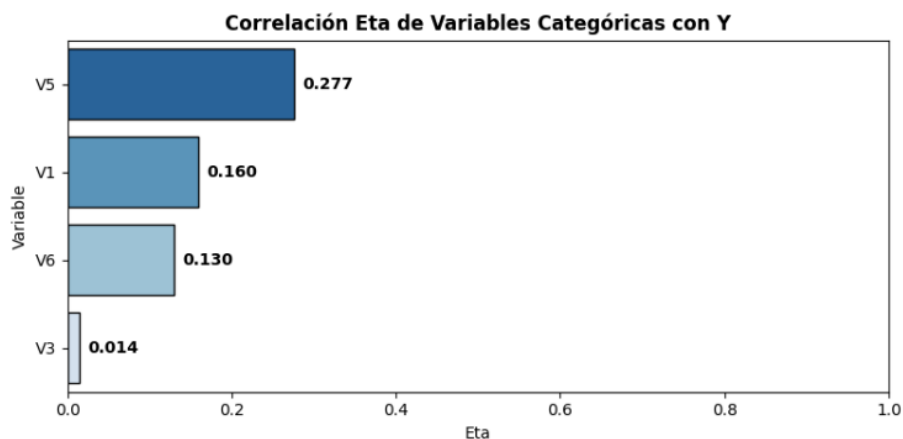
Detección estadística (IQR x 1.5):

V2: 22 outliers (0.62%)
V7: 150 outliers (4.24%)
V8: 19 outliers (0.54%)
V9: 27 outliers (0.76%)
V10: 20 outliers (0.57%)
V11: 31 outliers (0.88%)
V12: 29 outliers (0.82%)
V13: 31 outliers (0.88%)
V14: 32 outliers (0.90%)
V15: 24 outliers (0.68%)
V16: 31 outliers (0.88%)
V17: 49 outliers (1.38%)
V18: 93 outliers (2.63%)
V19: 49 outliers (1.38%)
V20: 326 outliers (9.21%)

Mediante IQR x 1.5, se detectan *outliers* en varias variables. El caso más llamativo es V20 (llamadas a soporte) con un 9,21% de *outliers* y V7 (mensajes buzón de voz) con un 4,24%. La decisión es conservarlos: en un problema de *churn*, un cliente que llama muchas veces a soporte no es un error de medición, es exactamente la señal que quiero que el modelo aprenda.

1.7. Relaciones entre variables y la target





Las variables que muestran una mayor correlación positiva con la fuga de clientes son:

- Número de llamadas al servicio de atención (V20): Un cliente que llama repetidamente al servicio de atención suele estar experimentando insatisfacción con el servicio.
- Plan internacional contratado (V5): Curiosamente, tener contratado este plan se asocia positivamente con el abandono.

En el extremo opuesto, encontramos variables con coeficientes negativos (a mayor valor, menos probabilidad de que el cliente se vaya):

- Plan de buzón de voz contratado (V6) y Número de mensajes de buzón de voz (V7): Los clientes que utilizan activamente estos servicios adicionales de la compañía muestran una menor tendencia a abandonar el servicio.

Las variables de consumo puro (V11, V12, V13, etc.) tienen correlaciones cercanas a cero de forma individual. Pero eso no quiere decir que no sean útiles para capturar nuevas interacciones.

1.8. Feature Engineering: creación de nuevas variables

```

1 df_fe = df.copy()
2
3 # Consumo agregado por franjas horarias
4 df_fe['Total_Minutos'] = df_fe['V8'] + df_fe['V11'] + df_fe['V14'] + df_fe['V17']
5 df_fe['Total_Llamadas'] = df_fe['V9'] + df_fe['V12'] + df_fe['V15'] + df_fe['V18']
6 df_fe['Total_Coste'] = df_fe['V10'] + df_fe['V13'] + df_fe['V16'] + df_fe['V19']
7
8 # Coste por minuto: percepción de valor del plan
9 df_fe['Coste_Por_Minuto'] = df_fe['Total_Coste'] / (df_fe['Total_Minutos'] + 1e-6)
10
11 # Ratio soporte/antigüedad: insatisfacción temprana
12 df_fe['Ratio_Soporte_Antigüedad'] = df_fe['V20'] / (df_fe['V2'] + 1)
13
14 nuevas = ['Total_Minutos', 'Total_Llamadas', 'Total_Coste',
15           'Coste_Por_Minuto', 'Ratio_Soporte_Antigüedad']
16 print("Correlación con Y de las nuevas variables:")
17 print(df_fe[nuevas].corrwith(df_fe['Y']).abs().sort_values(ascending=False).round(4))

```

Correlación con Y de las nuevas variables:

Total_Coste	0.2645
Total_Minutos	0.2310
Coste_Por_Minuto	0.1104
Ratio_Soporte_Antigüedad	0.0836
Total_Llamadas	0.0073
dtype:	float64

- `Total_Minutos / Total_Llamadas / Total_Coste`: Ayuda al modelo a identificar perfiles más sensibles a variaciones en la calidad del servicio.
- `Coste_Por_Minuto`: mide si el cliente percibe su tarifa como cara o barata. Las variables de coste y minutos por separado no capturan esta percepción de valor.
- `Ratio_Soporte_Antigüedad`: normaliza las llamadas a soporte (V20) por la antigüedad del contrato (V2). Un cliente nuevo con cinco llamadas a soporte es mucho más preocupante que uno veterano con el mismo número. Esta variable captura la insatisfacción temprana con más potencia que el conteo en bruto.

Eliminamos las variables originales involucradas. Se ha comprobado que las correlaciones individuales de V8–V19 con Y son cercanas a cero y además presentan altas correlaciones con los agregados. Mantener ambos conjuntos generaría redundancia que penaliza al SVM sin aportar capacidad discriminante adicional.

Correlación de cada variable original con su agregado:

```
Minutos (Total_Minutos):
V8 vs Total_Minutos: r = 0.6317
V11 vs Total_Minutos: r = 0.5608
V14 vs Total_Minutos: r = 0.5592
V17 vs Total_Minutos: r = 0.0132

Llamadas (Total_Llamadas):
V9 vs Total_Llamadas: r = 0.5722
V12 vs Total_Llamadas: r = 0.5725
V15 vs Total_Llamadas: r = 0.5799
V18 vs Total_Llamadas: r = 0.1002

Coste (Total_Coste):
V10 vs Total_Coste: r = 0.8893
V13 vs Total_Coste: r = 0.4174
V16 vs Total_Coste: r = 0.2311
V19 vs Total_Coste: r = 0.0520
```

1.9. Preprocesamiento final

Para V1 y V3 se aplica *One-Hot Encoding* con `drop_first=True` para evitar multicolinealidad perfecta. Las dimensiones finales del dataset son (3.538, 63). El *split* es estratificado 80/20: Train (2.830, 62) con 20.0% churn y Test (708, 62) con 19.9% churn. La estandarización se ajusta solo sobre train y se aplica a test sin recalibrar, para evitar *data leakage*.

V1 genera 50 variables indicadoras tras el *encoding*, lo que supone la mayor parte de las 62 variables del dataset. Para evaluar si estas variables aportan o perjudican, se compara un SVM RBF con parámetros por defecto usando el conjunto completo frente a uno reducido sin V1 ni V3 (10 variables). El reducido obtiene un AUC de 0.9074 frente al 0.8573 del completo. Esto indica que las 52 variables geográficas introducen ruido que perjudica al SVM. Por tanto, se decide utilizar el conjunto reducido de 10 variables para el SVM y el *Bagging*, ya que el *kernel* RBF calcula distancias euclídeas en todo el espacio de variables simultáneamente, y 50 *dummies* binarias dispersas diluyen la señal de las variables realmente discriminantes. El *Random Forest* y el *Stacking* utilizarán el conjunto completo de 62 variables, ya que el RF realiza selección de variables interna mediante sus particiones recursivas y no se ve perjudicado por la alta dimensionalidad.

```

1 # Comparación de conjuntos de variables: completo vs reducido (sin V1 y V3)
2 cols_estado = [c for c in X_train.columns if c.startswith("Estado_")]
3 cols_area = [c for c in X_train.columns if c.startswith("Area_")]
4 cols_reducido = [c for c in X_train.columns if c not in cols_estado + cols_area]
5
6 print(f"Conjunto completo: {X_train.shape[1]} variables")
7 print(f"Conjunto reducido (sin V1 ni V3): {len(cols_reducido)} variables")
8
9 svm_test = SVC(kernel="rbf", class_weight="balanced", random_state=42)
10 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
11
12 auc_completo = cross_val_score(svm_test, X_train_scaled, y_train,
13                               cv=skf, scoring="roc_auc").mean()
14
15 sc_red = StandardScaler()
16 X_train_red = sc_red.fit_transform(X_train[cols_reducido])
17 auc_reducido = cross_val_score(svm_test, X_train_red, y_train,
18                               cv=skf, scoring="roc_auc").mean()
19
20 print(f"AUC completo: {auc_completo:.4f}")
21 print(f"AUC reducido: {auc_reducido:.4f}")
22 print(f"Diferencia: {auc_completo - auc_reducido:.4f}")

```

Conjunto completo: 62 variables
 Conjunto reducido (sin V1 ni V3): 10 variables
 AUC completo: 0.8573
 AUC reducido: 0.9074
 Diferencia: -0.0500

2. Modelado con SVM - Comparativa de Kernels y Búsqueda de Hiperparámetros

Los datos están desbalanceados 80/20 tras eliminar duplicados. Para compensar el desbalanceo, usaremos `class_weight='balanced'` en los modelos, que pondera inversamente las clases sin duplicar datos. En esta situación el *accuracy* puede ser engañoso: un modelo que prediga siempre "no abandona" ya tiene un 80% de acierto sin haber aprendido nada. Por eso se usa el AUC como métrica principal, que mide la capacidad de discriminar entre clases independientemente del umbral. Además, desde el punto de vista del negocio, no detectar a un cliente que se va (falso negativo) es peor que ofrecer retención a uno que se queda (falso positivo), porque perder un cliente tiene un coste mayor que el de una llamada de retención innecesaria. Esto justifica prestar especial atención al *Recall* de la clase *churn*.

2.1. Búsqueda de hiperparámetros - Fase 1 (exploración amplia)

Para el SVM RBF hay que optimizar C (regularización) y gamma (radio de influencia del *kernel*) de forma conjunta, ya que están interrelacionados. Se hace una búsqueda en parrilla en dos fases.

```

1 # FASE 1: Grid amplio de exploración (sobre el conjunto REDUCIDO de 10 variables)
2 param_grid_fase1 = [
3     {'kernel': ['linear'], 'C': [0.01, 0.1, 1, 10, 100]},
4     {'kernel': ['rbf'], 'C': [0.1, 1, 10, 100],
5      'gamma': ['scale', 0.001, 0.01, 0.1]}
6 ]
7
8 svm = SVC(class_weight='balanced', probability=False, random_state=42)
9
10 grid_fase1 = GridSearchCV(
11     svm, param_grid_fase1, cv=3,
12     scoring={'AUC': 'roc_auc', 'Accuracy': 'accuracy'},
13     refit='AUC', verbose=1, n_jobs=-1
14 )
15 grid_fase1.fit(X_train_svm, y_train)
16
17 print(f"\nMejores parámetros (Fase 1): {grid_fase1.best_params_}")
18 print(f"AUC validación (Fase 1): {grid_fase1.best_score_:.4f}")
19

```

Fitting 3 folds for each of 21 candidates, totalling 63 fits
 Mejores parámetros (Fase 1): {'C': 1, 'gamma': 0.1, 'kernel': 'rbf'}
 AUC validación (Fase 1): 0.9072

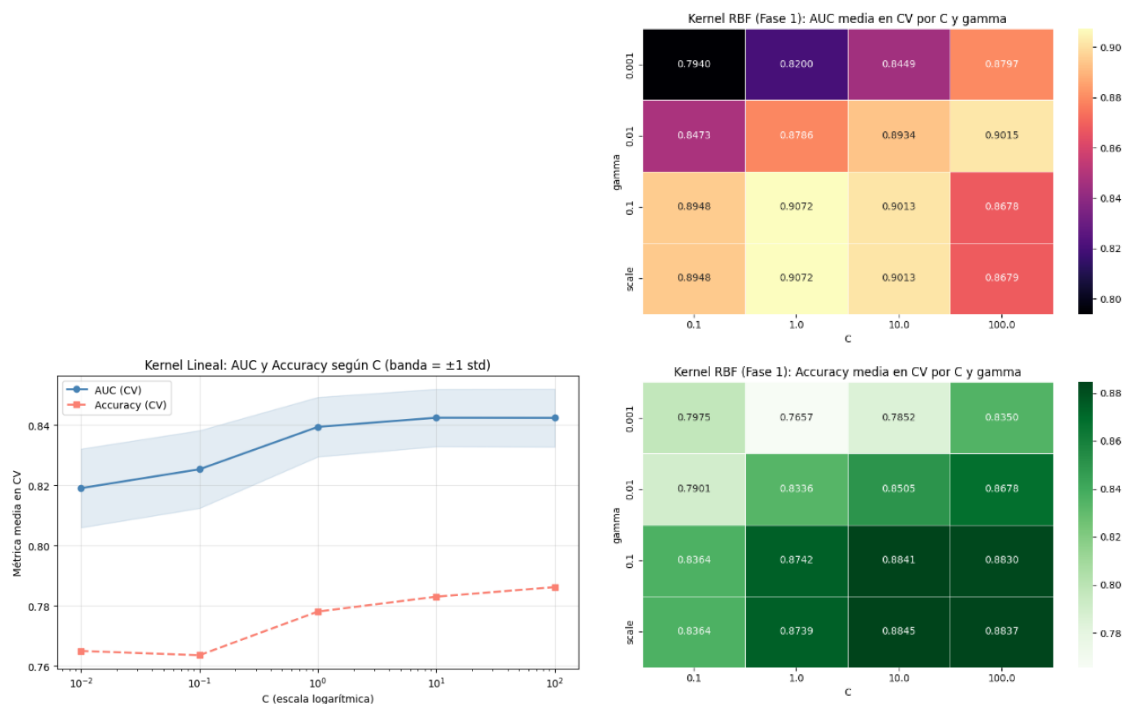
La Fase 1 es una exploración amplia con dos *kernels*:

- Lineal: $C = [0.01, 0.1, 1, 10, 100]$
- RBF: $C = [0.1, 1, 10, 100]$ x $\gamma = [\text{scale}, 0.001, 0.01, 0.1]$

Esto da $5 + 16 = 21$ combinaciones, evaluadas con $cv=3$, midiendo AUC y *Accuracy*, y *refit* por AUC.

Se utiliza $cv=3$ en esta fase exploratoria para reducir el coste computacional de evaluar 21 combinaciones, ya que el objetivo es descartar zonas claramente subóptimas del espacio de hiperparámetros.

El mejor resultado de la Fase 1 es RBF con $C=1$, $\gamma=0.1$ y AUC en validación de 0.9072.



En los heatmaps del RBF se ve claramente la interdependencia entre C y γ : la zona óptima se concentra en γ entre 0.05 y 0.1 con C moderados (1–5), mientras que valores extremos de γ (0.5 o 0.001) degradan el rendimiento independientemente de C .

2.2. Búsqueda de hiperparámetros - Fase 2 (refinamiento)

A partir del óptimo de la Fase 1 ($C=1$, $\gamma=0.1$), refino la búsqueda en su vecindad con un grid más fino y $cv=5$ folds para una estimación más robusta:

- $C = [0.25, 0.5, 1, 2, 5, 10]$
- $\gamma = [0.02, 0.05, 0.1, 0.2, 0.5]$

```

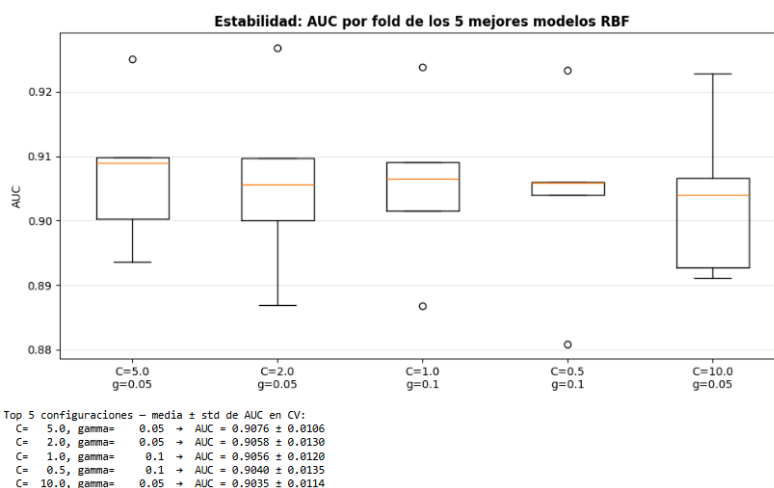
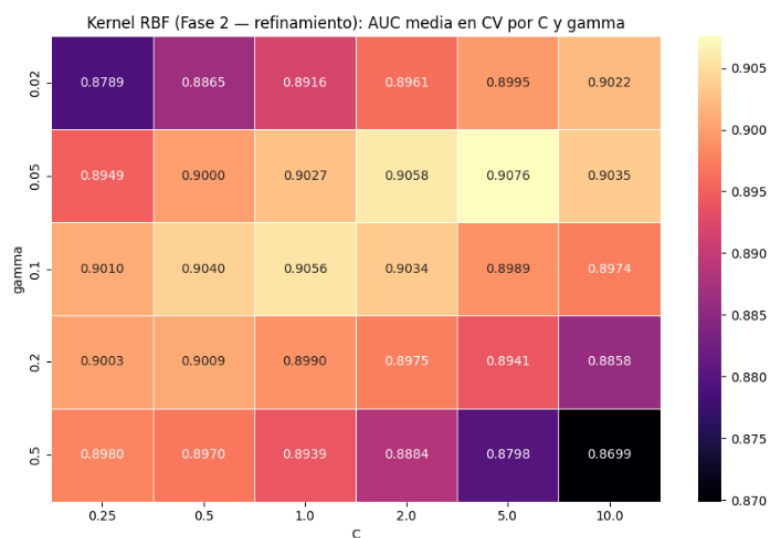
1 # FASE 2: Refinamiento en la zona óptima del RBF
2 best_C_f1 = grid_fase1.best_params_.get('C', 1)
3 best_g_f1 = grid_fase1.best_params_.get('gamma', 0.01)
4
5 # Construimos vecindad adaptativa
6 if isinstance(best_g_f1, str): # 'scale'
7     gamma_range = [0.001, 0.005, 0.01, 0.02, 0.05]
8 else:
9     gamma_range = sorted(set([round(best_g_f1 * f, 6) for f in [0.2, 0.5, 1, 2, 5]]))
10
11 C_range = sorted(set([round(best_C_f1 * f, 6) for f in [0.25, 0.5, 1, 2, 5, 10]]))
12
13 print(f"Zona de refinamiento basada en Fase 1 (C={best_C_f1}, gamma={best_g_f1}):")
14 print(f"  C      = {C_range}")
15 print(f"  gamma = {gamma_range}")
16
17 param_grid_fase2 = {
18     'kernel': ['rbf'],
19     'C': C_range,
20     'gamma': gamma_range
21 }
22
23 grid_fase2 = GridSearchCV(
24     SVC(class_weight='balanced', probability=False, random_state=42),
25     param_grid_fase2, cv=5,
26     scoring={'AUC': 'roc_auc', 'Accuracy': 'accuracy'},
27     refit='AUC', verbose=1, n_jobs=-1
28 )
29 grid_fase2.fit(X_train_svm, y_train)
30
31 print(f"\nMejores parámetros (Fase 2): {grid_fase2.best_params_}")
32 print(f"AUC validación (Fase 2): {grid_fase2.best_score_:.4f}")
33
34 best_grid = grid_fase2

```

Zona de refinamiento basada en Fase 1 (C=1, gamma=0.1):
 C = [0.25, 0.5, 1, 2, 5, 10]
 gamma = [0.02, 0.05, 0.1, 0.2, 0.5]
 Fitting 5 folds for each of 30 candidates, totalling 150 fits

Mejores parámetros (Fase 2): {'C': 5, 'gamma': 0.05, 'kernel': 'rbf'}
 AUC validación (Fase 2): 0.9076

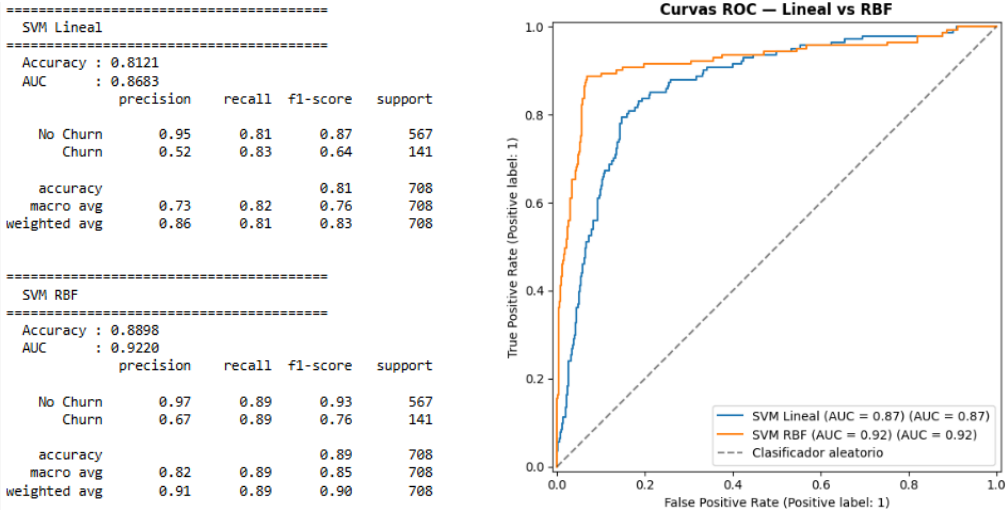
Esto da 30 combinaciones adicionales. El resultado tras refinar es C=5, gamma=0.05 con AUC en validación de 0.9076, una ligera mejora respecto al 0.9072 de la Fase 1.



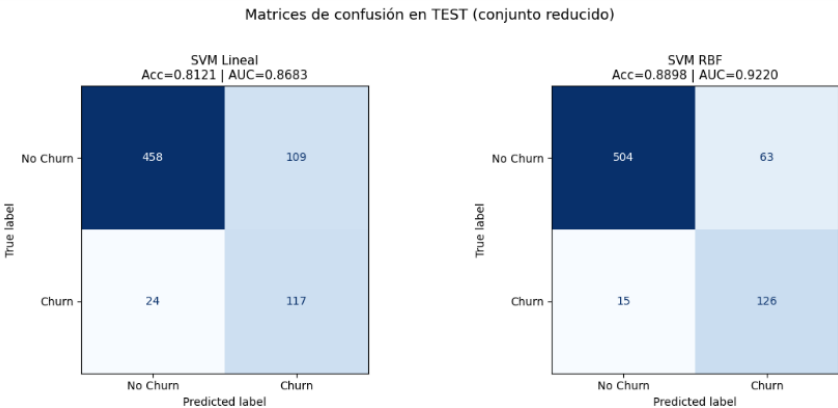
Las 5 mejores configuraciones muestran AUC robustos, con desviaciones inferiores a 0.015. La proximidad de estos valores y la baja dispersión entre *folds* confirman que el modelo es estable y que la elección del ganador no depende de un *fold* favorable concreto.

El que tiene el C más bajo es el cuarto, con C=0.5 y gamma=0.10. Su AUC es 0.9040 frente al 0.9076 del ganador, es decir pierde apenas 0.0036 de AUC. Esa diferencia está dentro de la variabilidad entre folds. Un C más bajo implica un modelo menos complejo. El parámetro C controla cuánto penalizamos las clasificaciones incorrectas en el entrenamiento: un C alto le dice al modelo "intenta clasificar bien cada punto, aunque la frontera sea muy retorcida", mientras que un C bajo le dice "está bien fallar en algunos puntos si con eso consigues una frontera con mayor margen". En la práctica, un C bajo amplía el margen de separación tolerando más puntos dentro del margen o mal clasificados, y eso produce una frontera de decisión más generalizable. Se podría elegir C=0.5 en un entorno de producción donde la parsimonia es el factor predominante.

2.3. Comparación de los dos mejores modelos



El SVM Lineal tiene un *recall* de 0.83 para la clase *Churn* pero una *precision* de solo 0.52: clasifica correctamente a la mayoría de los que se van, pero genera muchos falsos positivos. El SVM RBF mejora en todos los aspectos: *recall* de 0.89 para *Churn* con *precision* de 0.67, y un *accuracy* global del 88.98% frente al 81.21% del lineal.



El RBF saca 0.9076 en CV frente al 0.8413 del lineal, 7 puntos de diferencia que se mantienen en test (0.9220 vs 0.8683). El lineal no puede captar relaciones no lineales como el efecto que parece tener V20 sobre el *churn*, donde a partir de cierto número de llamadas la tasa de abandono sube de golpe. El RBF sí puede modelar ese tipo de saltos. Me quedo con el SVM RBF (C=5, gamma=0.05).

3. Ensamblado: Bagging y Stacking

3.1. Bagging

El *Bagging* construye múltiples réplicas del modelo base, cada una entrenada sobre una muestra con reemplazo (*bootstrap*) del conjunto de entrenamiento. Se utiliza muestreo con reemplazo (`'bootstrap=True'`, valor por defecto en `'BaggingClassifier'`) porque es más eficiente computacionalmente que sin reemplazo y, para datasets de este tamaño, ambas estrategias producen resultados prácticamente equivalentes.

Se ha probado desde 5 estimadores (mínimo para que el promediado tenga efecto) hasta 50 (más allá la mejora marginal suele desaparecer y el coste computacional crece). Para `max_samples` he probado 0.6 (submuestras más diversas entre sí), 0.8 (equilibrio) y 1.0 (máxima información por réplica, diversidad solo por el muestreo con reemplazo).

```
# Exploración de hiperparámetros del Bagging (sobre conjunto reducido, mismo que el SVM base)
print("Búsqueda de configuración del Bagging:\n")
resultados_bag = []
for n_est in [5, 10, 15, 25, 50]:
    for max_s in [0.6, 0.8, 1.0]:
        bag_temp = BaggingClassifier(
            estimator=best_svm,
            n_estimators=n_est,
            max_samples=max_s,
            random_state=42,
            n_jobs=-1
        )
        cv_auc = cross_val_score(bag_temp, X_train_svm, y_train,
                                cv=3, scoring='roc_auc').mean()
        resultados_bag.append({'n_estimators': n_est, 'max_samples': max_s, 'AUC_CV': cv_auc})
        print(f" n_est={n_est:3d}, max_samples={max_s:.1f} → AUC CV = {cv_auc:.4f}")

df_bag = pd.DataFrame(resultados_bag).sort_values('AUC_CV', ascending=False)
best_bag_row = df_bag.iloc[0]
print(f"\nMejor Bagging: n_est={int(best_bag_row['n_estimators'])}, "
      f"max_samples={best_bag_row['max_samples']:.1f}, AUC CV={best_bag_row['AUC_CV']:.4f}")

# Entrenar el Bagging final
bagging_model = BaggingClassifier(
    estimator=best_svm,
    n_estimators=int(best_bag_row['n_estimators']),
    max_samples=best_bag_row['max_samples'],
    random_state=42,
    n_jobs=-1
)
bagging_model.fit(X_train_svm, y_train)

y_pred_bag = bagging_model.predict(X_test_svm)
y_prob_bag = bagging_model.predict_proba(X_test_svm)[:, 1]
acc_bag = accuracy_score(y_test, y_pred_bag)
auc_bag = roc_auc_score(y_test, y_prob_bag)

print(f"\nBagging final en TEST - Accuracy: {acc_bag:.4f} | AUC: {auc_bag:.4f}")
print(classification_report(y_test, y_pred_bag, target_names=['No Churn', 'Churn']))
```

Búsqueda de configuración del Bagging:

```

n_est= 5, max_samples=0.6 → AUC CV = 0.9065
n_est= 5, max_samples=0.8 → AUC CV = 0.9078
n_est= 5, max_samples=1.0 → AUC CV = 0.9073
n_est= 10, max_samples=0.6 → AUC CV = 0.9083
n_est= 10, max_samples=0.8 → AUC CV = 0.9088
n_est= 10, max_samples=1.0 → AUC CV = 0.9090
n_est= 15, max_samples=0.6 → AUC CV = 0.9083
n_est= 15, max_samples=0.8 → AUC CV = 0.9089
n_est= 15, max_samples=1.0 → AUC CV = 0.9086
n_est= 25, max_samples=0.6 → AUC CV = 0.9084
n_est= 25, max_samples=0.8 → AUC CV = 0.9088
n_est= 25, max_samples=1.0 → AUC CV = 0.9091
n_est= 50, max_samples=0.6 → AUC CV = 0.9092
n_est= 50, max_samples=0.8 → AUC CV = 0.9091
n_est= 50, max_samples=1.0 → AUC CV = 0.9092

```

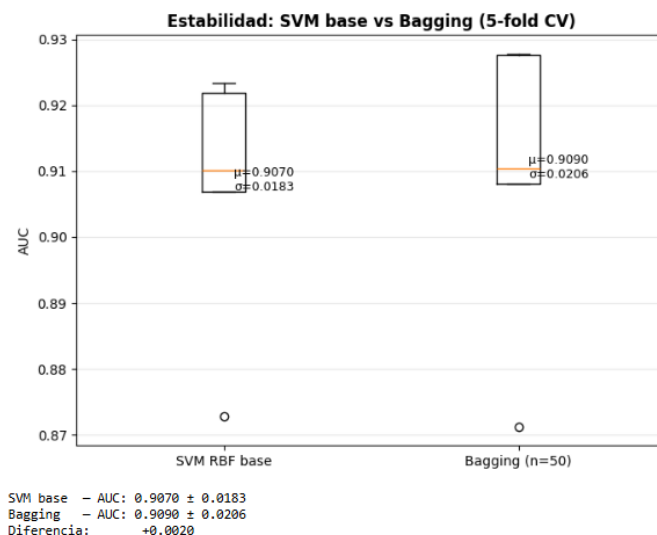
Mejor Bagging: n_est=50, max_samples=1.0, AUC CV=0.9092

Bagging final en TEST – Accuracy: 0.9025 AUC: 0.9203	
	precision recall f1-score support
No Churn	0.97 0.91 0.94 567
Churn	0.70 0.89 0.78 141
accuracy	0.90 708
macro avg	0.84 0.90 0.86 708
weighted avg	0.92 0.90 0.91 708

La mejor configuración es n_est=50, max_samples=1.0 con AUC CV=0.9092. En test: Accuracy=0.9025, AUC=0.9203.

El *Bagging* obtiene resultados prácticamente idénticos al SVM base. Esto teóricamente coherente: el *Bagging* funciona realmente bien cuando el modelo base tiene alta varianza (como un árbol de decisión sin podar). El SVM con C y gamma ya ajustados por validación cruzada es un modelo estable y de baja varianza; distintas submuestras *bootstrap* generan hiperplanos de separación muy parecidos.

Además, cada réplica del *Bagging* entrena con menos datos, lo que empeora ligeramente cada modelo individual. La pregunta es si la reducción de varianza al promediar compensa ese empeoramiento.



El *boxplot* de estabilidad confirma visualmente esta conclusión. El SVM base obtiene un AUC medio de 0.9070 ± 0.0183 en los 5 *folds* y el *Bagging* un 0.9090 ± 0.0206 , una diferencia de apenas 0.0020. Las cajas de ambos modelos se solapan casi por completo, lo que indica que la mejora no es estadísticamente significativa. Además, la desviación estándar del *Bagging* (0.0206) es incluso ligeramente superior a la del SVM (0.0183), es

decir, el *Bagging* no solo no mejora el rendimiento, sino que tampoco reduce la dispersión entre *folds*.

3.2. Stacking

El *Stacking* introduce un nivel adicional de abstracción. Varios clasificadores base heterogéneos generan predicciones sobre datos que no vieron durante su entrenamiento (predicciones *out-of-fold*). Esas predicciones se usan como nuevas variables de entrada para un meta-modelo que aprende la combinación óptima. La clave está en la diversidad algorítmica: si los modelos base cometen errores en casos distintos, el meta-modelo puede aprender a aprovechar las fortalezas de cada uno.

```

1 # Optimización del Random Forest (clasificador base del stacking)
2 param_rf = {
3     'n_estimators': [100, 200, 300],
4     'max_depth': [5, 10, 15, 20, None],
5     'min_samples_leaf': [1, 3, 5]
6 }
7
8 grid_rf = GridSearchCV(
9     RandomForestClassifier(class_weight='balanced', random_state=42),
10    param_rf, cv=3, scoring='roc_auc', n_jobs=-1
11 )
12 grid_rf.fit(X_train_scaled, y_train)
13 best_rf = grid_rf.best_estimator_
14
15 print(f"Mejor Random Forest: {grid_rf.best_params_}")
16 print(f"AUC en CV: {grid_rf.best_score_:.4f}")

```

Mejor Random Forest: {'max_depth': None, 'min_samples_leaf': 1, 'n_estimators': 100}
AUC en CV: 0.9173

```

1 # Top 5 configuraciones para análisis de estabilidad
2 res_rf = pd.DataFrame(grid_rf.cv_results_).sort_values('mean_test_score', ascending=False).head(5)
3 print("\nTop 5 configuraciones RF:")
4 for _, row in res_rf.iterrows():
5     print(f"  depth={str(row['param_max_depth']):>4}, leaf={row['param_min_samples_leaf']}, "
6           f"n_est={row['param_n_estimators']} → AUC = {row['mean_test_score']:.4f} ± {row['std_test_score']:.4f}")

```

Top 5 configuraciones RF:

depth=None, leaf=1, n_est=100	→ AUC = 0.9173 ± 0.0115
depth= 10, leaf=1, n_est=100	→ AUC = 0.9171 ± 0.0143
depth=None, leaf=1, n_est=200	→ AUC = 0.9170 ± 0.0102
depth= 10, leaf=1, n_est=200	→ AUC = 0.9170 ± 0.0125
depth= 10, leaf=5, n_est=100	→ AUC = 0.9170 ± 0.0122

```

1 # Evaluación del Random Forest en TEST (para cuantificar su contribución al stacking)
2 y_pred_rf = best_rf.predict(X_test_scaled)
3 y_prob_rf = best_rf.predict_proba(X_test_scaled)[: , 1]
4 acc_rf = accuracy_score(y_test, y_pred_rf)
5 auc_rf = roc_auc_score(y_test, y_prob_rf)
6 print(f"Random Forest en TEST - Accuracy: {acc_rf:.4f} | AUC: {auc_rf:.4f}")
7 print(classification_report(y_test, y_pred_rf, target_names=['No Churn', 'Churn']))
8

```

Random Forest en TEST - Accuracy: 0.9294 AUC: 0.9318				
	precision	recall	f1-score	support
No Churn	0.93	0.99	0.96	567
Churn	0.94	0.69	0.80	141
accuracy			0.93	708
macro avg	0.93	0.84	0.88	708
weighted avg	0.93	0.93	0.93	708

El *Random Forest*, evaluado individualmente en test, obtiene *Accuracy* = 0.9294 y *AUC* = 0.9318. Este resultado servirá de referencia para cuantificar cuánto aporta el mecanismo de *stacking*.

```

1 # Optimización mínima de La Regresión Logística base
2 print("Exploración de C para la Regresión Logística base:\n")
3 for c_lr in [0.01, 0.1, 1, 10, 25, 50, 100]:
4     lr_temp = LogisticRegression(C=c_lr, class_weight='balanced',
5                                 max_iter=1000, random_state=42)
6     cv_lr = cross_val_score(lr_temp, X_train_scaled, y_train,
7                             cv=3, scoring='roc_auc').mean()
8     print(f" C={c_lr}: AUC CV = {cv_lr:.4f}")

```

Exploración de C para la Regresión Logística base:

```

C=0.01: AUC CV = 0.8101
C=0.1: AUC CV = 0.8125
C=1: AUC CV = 0.8241
C=10: AUC CV = 0.8312
C=25: AUC CV = 0.8315
C=50: AUC CV = 0.8317
C=100: AUC CV = 0.8317

```

```

1 # Selección del mejor C para La LR base
2 best_C_lr = 50
3 best_lr = LogisticRegression(C=best_C_lr, class_weight='balanced',
4                             max_iter=1000, random_state=42)
5
6 # Clasificadores base del Stacking
7 # NOTA: El StackingClassifier pasa las mismas X a todos los estimadores base.
8 # Usamos el conjunto completo porque el RF (que domina el stacking) lo aprovecha.
9 # Se redefine el SVM base para el stacking con los mismos hiperparámetros óptimos.
10 svm_for_stack = SVC(**best_grid.best_params_,
11                    class_weight='balanced', probability=False, random_state=42)
12
13 base_estimators = [
14     ('svm', svm_for_stack),
15     ('rf', best_rf),
16     ('lr', best_lr)
17 ]
18
19 # Optimización del meta-modelo
20 print("\nBúsqueda de regularización para el meta-modelo:\n")
21 for c_meta in [0.01, 0.1, 1, 10]:
22     meta = LogisticRegression(C=c_meta, max_iter=500, random_state=42)
23     stack_temp = StackingClassifier(
24         estimators=base_estimators,
25         final_estimator=meta,
26         cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
27         n_jobs=-1
28     )
29     cv_auc = cross_val_score(stack_temp, X_train_scaled, y_train,
30                             cv=3, scoring='roc_auc').mean()
31     print(f" C={c_meta}: AUC CV = {cv_auc:.4f}")
32

```

Búsqueda de regularización para el meta-modelo:

```

C=0.01: AUC CV = 0.8863
C=0.1: AUC CV = 0.9195
C=1: AUC CV = 0.9174
C=10: AUC CV = 0.9144

```

Se probaron distintos valores de regularización para el meta-modelo (C= 0.01, 0.1, 1, 10). El mejor resultado se obtuvo con C=0.1, que es el que se usa en el modelo final.

```

1 meta_model = LogisticRegression(C=0.1, max_iter=500, random_state=42)
2
3 stacking_model = StackingClassifier(
4     estimators=base_estimators,
5     final_estimator=meta_model,
6     cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42),
7     n_jobs=-1
8 )
9 stacking_model.fit(X_train_scaled, y_train)
10
11 y_pred_stack = stacking_model.predict(X_test_scaled)
12 y_prob_stack = stacking_model.predict_proba(X_test_scaled)[: , 1]
13 acc_stack = accuracy_score(y_test, y_pred_stack)
14 auc_stack = roc_auc_score(y_test, y_prob_stack)
15
16 print(f"Stacking - Accuracy: {acc_stack:.4f}")
17 print(f"Stacking - AUC: {auc_stack:.4f}")
18 print(classification_report(y_test, y_pred_stack, target_names=['No Churn', 'Churn']))
19

```

```

Stacking - Accuracy: 0.9280
Stacking - AUC: 0.9253

```

	precision	recall	f1-score	support
No Churn	0.93	0.98	0.96	567
Churn	0.91	0.71	0.80	141
accuracy			0.93	708
macro avg	0.92	0.85	0.88	708
weighted avg	0.93	0.93	0.92	708

Se seleccionan tres clasificadores base que aporten diversidad algorítmica: el SVM RBF ($C=5$, $\gamma=0.05$) obtenido en el apartado anterior, un *Random Forest* (100 árboles, $\text{max_depth}=\text{None}$, $\text{min_samples_leaf}=1$, optimizado con `GridSearchCV`, $\text{AUC CV} = 0.9173$) cuya lógica de particiones recursivas es estructuralmente diferente al SVM, y una Regresión Logística ($C=50$, $\text{class_weight}=\text{'balanced'}$) que aporta una perspectiva lineal global complementaria a los dos modelos más complejos.

El meta-modelo es una Regresión Logística con $C=0.1$ (regularización moderada para evitar que copie siempre al modelo más potente), con predicciones *out-of-fold* generadas mediante `StratifiedKFold` de 5 *splits*.

El *Stacking* obtiene un *Accuracy* del 92.80% y un AUC de 0.9253, con *Recall* de 0.71 y *Precision* de 0.91 para la clase Churn. Se acerca al Random Forest individual en *Accuracy*, aunque queda por debajo en AUC (0.9253 vs 0.9318). La principal diferencia está en el equilibrio *Precision-Recall*: detecta ligeramente más clientes en riesgo (0.71 vs 0.69) a costa de algo más de falsos positivos (0.91 vs 0.94).

No obstante, el hecho de que el *Stacking* no supere al RF en AUC (0.9253 vs 0.9318) indica que el mecanismo de combinación no está aportando capacidad discriminatoria adicional. La ligera mejora en *Recall* no proviene de una mayor capacidad predictiva, sino de una redistribución del umbral de decisión. Para entender por qué, examinamos los coeficientes del meta-modelo.

```
1 # Coeficientes del meta-modelo: ¿cuánto pesa cada clasificador base?
2 meta = stacking_model.final_estimator_
3 nombres_base = [name for name, _ in stacking_model.estimators]
4
5 print("Pesos del meta-modelo (Regresión Logística):\n")
6 for name, coef in zip(nombres_base, meta.coef_[0]):
7     print(f" {name:>5s}: {coef:+.4f}")
8
9 print(f" {'Inter':>5s}: {meta.intercept_[0]:+.4f}")
10
```

Pesos del meta-modelo (Regresión Logística):

svm:	+0.3830
rf:	+4.4010
lr:	+1.0018
Inter:	-2.8610

Los coeficientes del meta-modelo confirman esta sospecha: el *Random Forest* domina la decisión final con un peso de +4.40, frente al +0.38 del SVM y el +1.00 de la Regresión Logística. En la práctica, el meta-modelo ha aprendido que la estrategia óptima es confiar principalmente en las predicciones del RF, con la LR como segundo contribuyente y el SVM como corrector menor.

El SVM dentro del *stacking* no está trabajando en sus condiciones óptimas y esto es una limitación de mi diseño. En el análisis previo demostré que el SVM RBF rinde mucho mejor con 10 variables ($\text{AUC} = 0.9074$) que con 62 ($\text{AUC} = 0.8573$). Sin embargo, el `StackingClassifier` pasa las mismas X a todos los clasificadores base, así que el SVM dentro del *stacking* recibe las 62 variables, no las 10 con las que rinde mejor.

Lo que ocurre en la práctica es que el meta-modelo se da cuenta de que el SVM no aporta mucho en esas condiciones: su peso es de solo +0.38 frente al +4.40 del Random Forest y +1.00 de la Regresión Logística.

4. Comparación Global de Modelos

4.1. Tabla resumen de resultados en test

Modelo	Accuracy	AUC	Precision_Churn	Recall_Churn	F1_Churn
SVM Lineal (10v)	0.8121	0.8683	0.5177	0.8298	0.6376
SVM RBF (10v)	0.8898	0.9220	0.6667	0.8936	0.7636
Bagging+SVM (10v)	0.9025	0.9203	0.7022	0.8865	0.7837
Random Forest (62v)	0.9294	0.9318	0.9417	0.6879	0.7951
Stacking (62v)	0.9280	0.9253	0.9091	0.7092	0.7968

El *Random Forest* y el *Stacking* se sitúan claramente por encima del resto, con el RF ligeramente por delante (RF: 0.9294/0.9318, *Stacking*: 0.9280/0.9253). El *Bagging* + SVM mejora al SVM RBF en *Accuracy* (0.9025 vs 0.8898) pero obtiene un AUC ligeramente inferior (0.9203 vs 0.9220), confirmando que esta técnica no aporta valor sobre un modelo ya estable.

Un aspecto relevante es el *trade-off* entre *Precision* y *Recall* de la clase *Churn*: el *Random Forest* tiene una *Precision* muy alta (0.94) pero un *Recall* más bajo (0.69), lo que significa que cuando identifica un cliente en riesgo, casi siempre acierta, pero deja sin detectar un 31% de los abandonos reales. El *Stacking* equilibra ligeramente esta relación (*Precision* 0.91, *Recall* 0.71), detectando un 2% más de clientes en riesgo a costa de algunos falsos positivos adicionales.

4.2. Validación cruzada de robustez

Para asegurar que los resultados no dependen de una partición favorable, evalúo todos los modelos con validación cruzada estratificada de 5 *folds* sobre los datos de entrenamiento:

```
1 # Validación cruzada de robustez
2 # SVM y Bagging se evalúan con el conjunto reducido; RF y Stacking con el completo
3 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
4
5 modelos_cv_config = {
6     'SVM Lineal (10v)': (best_svm_lin, X_train_svm),
7     'SVM RBF (10v)': (best_svm, X_train_svm),
8     'Bagging + SVM (10v)': (bagging_model, X_train_svm),
9     'Random Forest (62v)': (best_rf, X_train_scaled),
10    'Stacking (62v)': (stacking_model, X_train_scaled),
11 }
12
13 resultados_cv = []
14 nombres_modelos = []
15 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
16
17 for nombre, (modelo, X_data) in modelos_cv_config.items():
18     cv_scores = cross_val_score(modelo, X_data, y_train,
19                                 cv=skf, scoring='roc_auc', n_jobs=-1)
20     resultados_cv.append(cv_scores)
21     nombres_modelos.append(nombre)
22     print(f"{nombre}:")
23     print(f"    AUC media: {cv_scores.mean():.4f} | Std: {cv_scores.std():.4f}")
24     print()
25
```

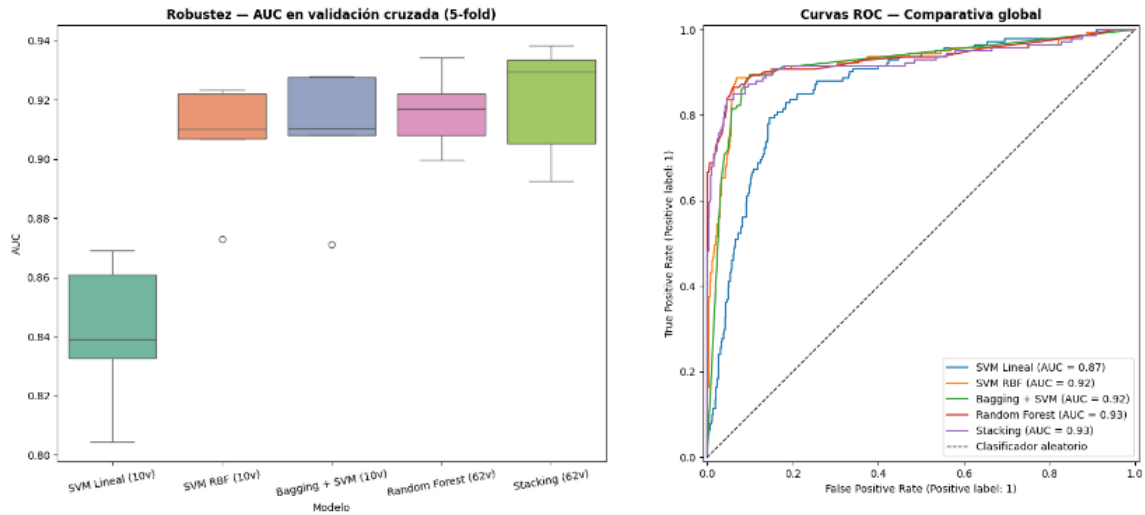
SVM Lineal (10v):
AUC media: 0.8413 | Std: 0.0228

SVM RBF (10v):
AUC media: 0.9070 | Std: 0.0183

Bagging + SVM (10v):
AUC media: 0.9090 | Std: 0.0206

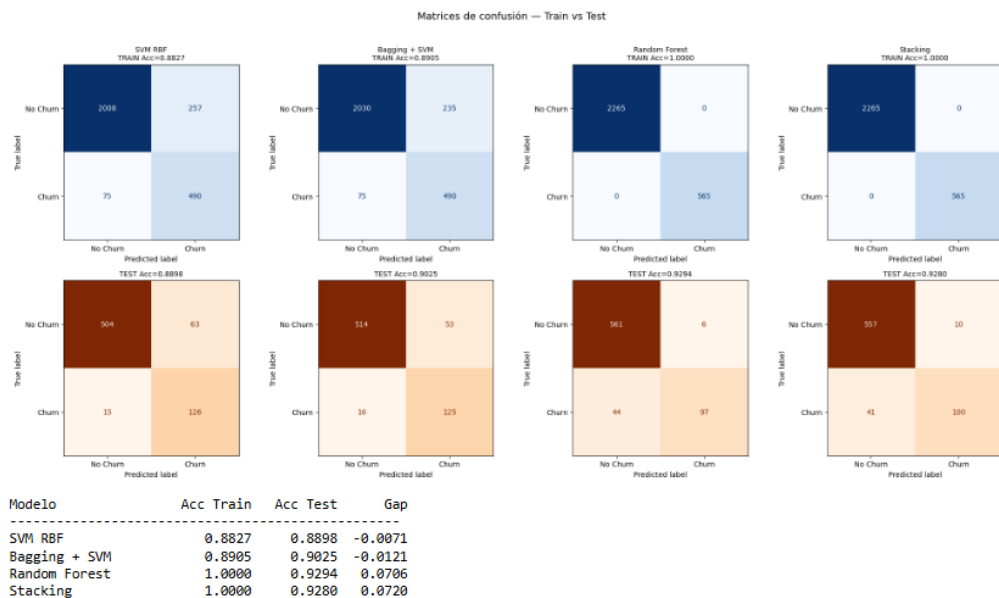
Random Forest (62v):
AUC media: 0.9162 | Std: 0.0119

Stacking (62v):
AUC media: 0.9198 | Std: 0.0178



El *Random Forest* y el *Stacking* encabezan los resultados con AUC medias de 0.9162 y 0.9198 respectivamente, aunque la ventaja sobre el SVM RBF (0.9070) y el *Bagging* (0.9090) es moderada. El único modelo que queda claramente rezagado es el SVM Lineal (0.8413). La diferencia entre RF y *Stacking* es de apenas 0.004, lo que indica que la ganancia del *stacking* sobre el RF individual es mínima en términos de capacidad discriminatoria generalizada.

4.3. Modelo ganador: matrices de confusion (train vs test)



Modelo	TN	FP	FN	TP	Recall Churn	FP (%)	FN (%)
SVM Lineal (10v)	458	109	24	117	83.0%	19.2%	17.0%
SVM RBF (10v)	504	63	15	126	89.4%	11.1%	10.6%
Bagging+SVM (10v)	514	53	16	125	88.7%	9.3%	11.3%
Random Forest (62v)	561	6	44	97	68.8%	1.1%	31.2%
Stacking (62v)	557	10	41	100	70.9%	1.8%	29.1%

El SVM RBF y el *Bagging* presentan un *gap* negativo entre train y test (-0.0071 y -0.0121 respectivamente), lo que indica ausencia de *overfitting*. En la tabla de contingencia, ambos destacan por su alto *recall* de la clase *churn*: el SVM RBF detecta 126 de los 141 clientes que abandonan (89.4%) y el *Bagging* 125 (88.7%), a costa de generar más falsos positivos (63 y 53 respectivamente, un 11.1% y 9.3% de los clientes que no abandonan). Es decir, estos modelos priorizan no dejar escapar clientes en riesgo, aunque eso implique contactar a algunos que realmente no se iban a ir.

El *Random Forest* y el *Stacking* muestran el patrón opuesto. Ambos tienen *accuracy* perfecta en train (1.0) con un *gap* de 7 puntos respecto a test. En el caso del RF es esperable: sin limitar *max_depth*, el algoritmo genera hojas puras para cada observación y memoriza todo el entrenamiento. El *Stacking* hereda este comportamiento porque el RF es su clasificador dominante. Sin embargo, donde realmente hay diferencia es en la tabla de contingencia. El *Random Forest* solo genera 6 falsos positivos (1.1% de los *no-churn*), pero se le escapan 44 de los 141 clientes que abandonan (31.2%). El *Stacking* mejora ligeramente este equilibrio: detecta 100 de 141 (70.9%) con 10 falsos positivos (1.8%).

La tabla pone de manifiesto un *trade-off* claro que las métricas agregadas no capturan del todo. El SVM RBF detecta 126 *churns* pero lanza 63 alertas innecesarias. El *Random Forest* solo lanza 6 alertas innecesarias, pero deja escapar 44 *churns*. La elección depende del contexto de negocio: si el coste de perder un cliente es muy superior al de una llamada de retención innecesaria, el perfil del SVM RBF es preferible; si los recursos de retención son limitados y se necesita que cada alerta sea fiable, el *Random Forest* es más adecuado. El *Stacking* se sitúa como punto intermedio, con 100 *churns* detectados y solo 10 falsos positivos, ofreciendo el mejor equilibrio entre ambos extremos.

4.4. Robustez con distintas semillas

Como verificación adicional, se reentrena el SVM RBF, el *Random Forest* y el *Stacking* con 5 semillas distintas de split train/test para comprobar que los resultados no dependen de una partición favorable:

Robustez con distintas semillas de split:

Seed 42:	SVM=0.9220	RF=0.9318	Stacking=0.9253
Seed 123:	SVM=0.9238	RF=0.9235	Stacking=0.9292
Seed 456:	SVM=0.8912	RF=0.8928	Stacking=0.9016
Seed 789:	SVM=0.9008	RF=0.9116	Stacking=0.9174
Seed 2024:	SVM=0.9007	RF=0.9162	Stacking=0.9175

SVM RBF (10v)	- AUC medio: 0.9077 ± 0.0129	Rango: [0.8912, 0.9238]
RF (62v)	- AUC medio: 0.9152 ± 0.0131	Rango: [0.8928, 0.9318]
Stacking (62v)	- AUC medio: 0.9182 ± 0.0095	Rango: [0.9016, 0.9292]

Los tres modelos muestran un comportamiento consistente en las 5 particiones. El *Random Forest* y el *Stacking* superan al SVM RBF de forma sistemática, con AUC medios de 0.9152 y 0.9182 respectivamente frente al 0.9077 del SVM. La dispersión del *Stacking* es incluso menor que la del resto (± 0.0095 frente a ± 0.0131 del RF y ± 0.0129 del SVM), lo que indica mayor estabilidad. El *Stacking* obtiene un AUC medio ligeramente superior al RF (0.9182 vs 0.9152), y su peor caso (0.9016) también supera al

peor caso del RF (0.8928), lo que sugiere una ligera ventaja del ensamblado en robustez. Aun así, la diferencia es mínima y ambos modelos mantienen un rendimiento sólido incluso en la partición menos favorable.

4.5. Conclusiones

En este apartado se discuten los resultados obtenidos por los diferentes algoritmos, evaluando sus ventajas, inconvenientes y comportamiento:

- SVM Lineal: Es el modelo más sencillo e interpretable. Sus coeficientes permiten ver directamente el peso de cada variable. Pero su gran limitación es que no puede capturar las interacciones no lineales presentes en los datos, lo que le pone un techo de rendimiento claro (AUC de 0.84 en CV, 0.87 en test).
- SVM RBF ($C=5$, $\gamma=0.05$): Elimina el sesgo de linealidad a costa de perder interpretabilidad directa. AUC de 0.92 en test, robusto en validación cruzada (0.9070 ± 0.0183) y ante cambios de semilla (0.9077 ± 0.0129). La selección del conjunto reducido de 10 variables (sin V1 ni V3) ha sido clave para alcanzar este rendimiento, ya que el mismo modelo con 62 variables obtenía un AUC de apenas 0.86 en CV.
- *Bagging* + SVM: No aporta mejora significativa. Tras explorar 15 combinaciones de *n_estimators* y *max_samples*, el mejor *Bagging* obtiene un AUC ligeramente inferior al SVM individual (0.9203 vs 0.9220) y mejora la *Accuracy* (0.9025 vs 0.8898). El SVM ya es un modelo de baja varianza tras la regularización, así que el *Bagging* no tiene el problema que está diseñado para resolver. La mejora en *Accuracy* se debe a que el promedio de múltiples réplicas suaviza la frontera de decisión, reduciendo los falsos positivos (*Precision Churn* de 0.70 vs 0.67), pero sin mejorar la capacidad discriminatoria global medida por el AUC.
- *Random Forest*: Es el mejor modelo individual del estudio, con AUC de 0.9318 y *Accuracy* de 92.94% en test. Supera al SVM RBF sin necesidad de ensamblado adicional. Su *Precision* para la clase *Churn* es excepcionalmente alta (0.94), aunque a costa de un *Recall* más moderado (0.69).
- *Stacking* (SVM + RF + LR): Obtiene un *Accuracy* del 92.80% y un AUC de 0.9253, cercano al RF (92.94% y 0.9318). La principal diferencia está en el equilibrio *Precision-Recall* de la clase *Churn*: el *Stacking* sacrifica algo de *Precision* (0.91 vs 0.94) para ganar ligeramente en *Recall* (0.71 vs 0.69), detectando un 2% más de clientes en riesgo de abandono. Esto indica que el mecanismo de combinación aporta una redistribución del umbral de decisión más que una mejora en capacidad discriminatoria.

En este problema concreto, los métodos de ensamblado no justifican su complejidad adicional. El *Bagging* no mejora al SVM porque el SVM ya tiene baja varianza. El *Stacking* no supera al Random Forest porque el RF ya domina la combinación (peso de +4.40 en el meta-modelo frente a +0.38 del SVM y +1.00 de la LR). La lección práctica es que el ensamblado no es una mejora automática: su valor depende de que exista un problema (alta varianza, errores complementarios entre modelos) que la técnica pueda resolver. Cuando el modelo base ya es robusto o cuando un clasificador individual ya cubre bien el espacio, añadir complejidad no compensa.

En un entorno productivo, el Random Forest sería la opción más recomendable: ofrece el mejor AUC individual, es más sencillo de mantener y desplegar que un *Stacking* de tres modelos, su rendimiento es robusto en validación cruzada (0.9162 ± 0.0119) y ante distintas particiones (0.9152 ± 0.0131), y su coste computacional es menor. El *Stacking* solo se justificaría si el negocio priorizase detectar el mayor número posible de clientes en riesgo (*Recall*), aceptando más falsos positivos a cambio, ya que su *Recall* (0.71) supera ligeramente al del RF (0.69).

Dicho esto, la elección del modelo final depende del contexto de negocio. Si la empresa necesita explicar al cliente por qué le hace una oferta de retención, ni el Random Forest ni el *Stacking* permiten hacerlo de forma transparente. En esos casos, un modelo más sencillo como el SVM Lineal o la Regresión Logística, a pesar de sus métricas inferiores, ofrecería coeficientes interpretables que justifican la decisión ante el cliente o ante un regulador.